

Technical Implementation Document

Data Pipeline, System Architecture, and Training Workflow (Tacotron2-VAE & WaveGlow)

ml2_final_project

June 12, 2026

Abstract

This document provides a highly detailed, faithful representation of the `ml2_final_project` repository, specifically from the `merge/preparing_merge_vae_tacotron` branch. It thoroughly documents the repository structure, the complete data preprocessing pipeline, the progressively detailed neural network architectures of the Tacotron2-VAE acoustic model and the WaveGlow neural vocoder, as well as the exact loss functions, training mechanics, and configuration hyperparameters exactly as they are defined in the codebase.

Contents

1	Repository Structure and Overview	3
1.1	Top-Level Directory Tree	3
1.2	Directory Roles	3
2	Data Pipeline and Preprocessing	4
2.1	Phase 1: Offline Preprocessing	4
2.2	Phase 2: Online Dataset Loading	4
3	Model Architectures	6
3.1	5.1 Complete System Overview	6
3.2	5.2 Tacotron-VAE Module Overview	6
3.3	5.3 Text Encoder Breakdown	7
3.4	5.4 VAE (Prosody) Breakdown	7
3.5	5.5 Location-Sensitive Attention Breakdown	9
3.6	5.6 Autoregressive Decoder Breakdown	10
3.7	5.7 PostNet Breakdown	10
3.8	5.8 WaveGlow Vocoder Overview	12
3.9	5.9 WaveGlow Flow Step Breakdown	12
3.10	5.10 Layer-by-Layer Dimension Summary	12
4	Loss Functions	13
4.1	Tacotron2-VAE Losses (<code>losses.py</code>)	13
4.1.1	1. Reconstruction Loss ($\mathcal{L}_{\text{Recon}}$)	13
4.1.2	2. Gate Loss ($\mathcal{L}_{\text{Gate}}$)	13
4.1.3	3. Kullback-Leibler Divergence ($\mathcal{L}_{\text{KL}}$)	13

4.1.4	4. KL Annealing Strategy	13
4.2	WaveGlow Losses (<code>glow.py</code>)	14
5	Training Pipeline	15
5.1	Tacotron2-VAE Training	15
5.2	WaveGlow Training	15
6	Hyperparameters and Configurations	16

1 Repository Structure and Overview

The repository organizes its workflow into isolated components, ensuring that raw data, preprocessing scripts, model definitions, and training loops operate independently.

1.1 Top-Level Directory Tree

```
ml2_final_project/
|-- data/
|   |-- raw/                                # Original, unmodified audio corpora
|   |   |-- libriSpeech-pt/
|   |   |-- libriSpeech-en/
|   |   |-- tts-portuguese-Corpora/
|   |-- processed/                          # Serialized .pt tensors and metadata
|   |   |-- libriSpeech-en-tacotron-vae/
|   |   |-- tts_portuguese/
|-- scripts/
|   |-- download_datasets/                  # Shell/Python fetchers
|   |-- inference/                          # Audio generation utilities
|   |-- preprocess/                         # Normalization & feature extraction
|   |   |-- preprocess_libriSpeech-pt.py
|   |   |-- preprocess_libri-Speech-en_vaeTacotron.py
|   |   |-- preprocess_mels_tts_portuguese.py
|   |-- utils/                             # Model analysis scripts
|-- src/
|   |-- data/                              # PyTorch Datasets and DataLoaders
|   |   |-- loader_vae_tacotron/
|   |   |-- loader_waverglow/
|   |-- models/                            # Neural Network Architecture
|   |   |-- tacotron2_vae/                  # Acoustic model components
|   |   |-- waverglow/                      # Neural Vocoder components
|   |-- training/                           # Checkpointing, loops, and losses
|   |   |-- training-tacotron2-vae/
|   |   |-- training-waverglow/
|-- experiments/                           # Output artifacts, logs, checkpoints
|-- docs/                                  # Project documentation
|-- notebooks/                             # Jupyter exploratory analysis
|-- remote_access/                         # SSH automation/configs
```

1.2 Directory Roles

- **data/**: The lifecycle of audio starts in **data/raw/** and transitions to **data/processed/** via the preprocessing scripts.
- **scripts/**: Houses standalone execution scripts. Preprocessing scripts iterate over raw corpora, standardize sample rates, extract initial acoustic features, and write manifest CSVs.
- **src/models/**: The core PyTorch implementation. Divided explicitly into **tacotron2_vae** (handling text-to-mel logic) and **waverglow** (handling mel-to-audio logic).
- **src/training/**: Contains isolated training loops. Tacotron uses a standard iteration-based loop, while WaveGlow implements Distributed Data Parallel (DDP).

2 Data Pipeline and Preprocessing

The data pipeline bridges the gap between raw datasets and PyTorch `DataLoader` iterations. The pipeline consists of two primary phases: off-line preprocessing (`scripts/preprocess`) and on-line data loading (`src/data`).

2.1 Phase 1: Offline Preprocessing

The repository provides specialized scripts for different datasets (e.g., `preprocess_libri-Speech-en_vaeTacotron`).

Workflow sequence:

1. **Discovery:** The script recursively glob-searches for `*.trans.txt` files.
2. **Text Loading:** Each row matches an utterance ID to a transcription text.
3. **Audio Normalization:** The corresponding audio file is loaded via `torchaudio`. It is converted to mono (by averaging channels if necessary) and resampled to the target sampling rate (16000 Hz for EN, 22050 Hz for PT).
4. **Feature Extraction:** A specialized transform computes the 80-bin Mel-spectrogram. For Tacotron2-VAE compatibility, the `Tacotron2MelTransform` utilizes `librosa.filters.mel` to guarantee parity with the original Tacotron codebase.
5. **Duration Filtering:** Utterances shorter than 0.3s or longer than 10.0s (to prevent OOM errors) are discarded.
6. **Serialization:** The waveform, mel-spectrogram, text, duration, and sampling rate are serialized into a `.pt` file.
7. **Manifest Generation:** A `mels_metadata.csv` is generated containing `mel_path`, `duration`, `text`, `utt_id`.

Preprocessing Hyperparameters (LibriSpeech-EN):

Parameter	Value
Target Sample Rate	16000
Number of Mel Bins	80
FFT / Filter Length	1024
Window Length	1024
Hop Length	256
F-Min / F-Max	0.0 / 8000.0
Dynamic Range Compression Limit	1×10^{-5}

2.2 Phase 2: Online Dataset Loading

In `src/data/loader_vae_tacotron/loader_tacotron.py`, the `DatasetLibriSpeechTacotronVAE` class reads the generated `mels_metadata.csv`.

Crucial Implementation Detail: Although the offline scripts extract Mel-spectrograms, the `loader_tacotron.py` *ignores* the serialized Mel-spectrogram and re-computes it on the fly during the `__getitem__` call.

```
# Excerpt from loader_tacotron.py
def get_mel(self, audio):
    audio_norm = audio / MAX_WAV_VALUE
    audio_norm = audio_norm.unsqueeze(0)
    audio_norm = torch.autograd.Variable(audio_norm, requires_grad=False)
    melspec = self.stft.mel_spectrogram(audio_norm)
    return torch.squeeze(melspec, 0)
```

Online STFT Configuration: The online STFT relies on TacotronSTFT defined in `layers.py`, configured specifically with `filter_length=800`, `hop_length=200`, `win_length=800`, `sampling_rate=22050`.

Text Normalization: The text is passed through `TextNormalizerEN`, which utilizes Regex to strip punctuation and the `num2words` library to spell out integer values before tokenization.

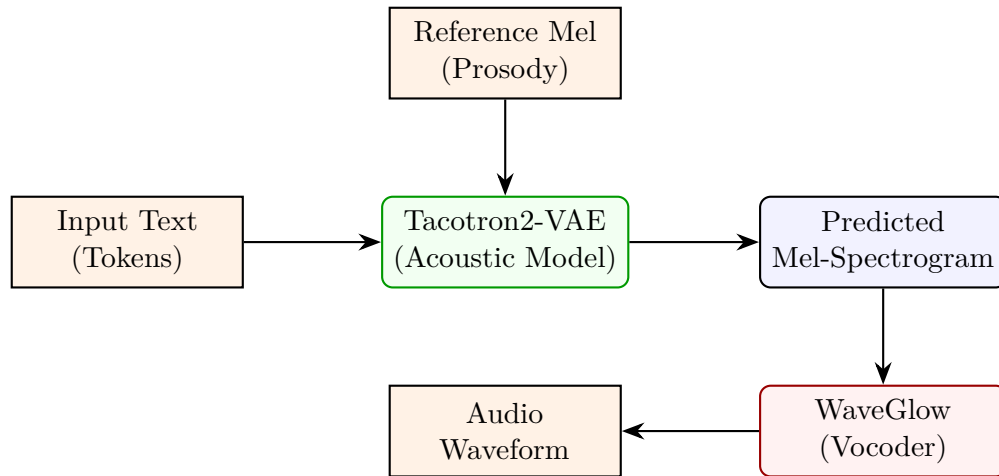
Emotion Tensor Injection: An emotion tensor of shape [4] is initialized and attached to every batch. By default, index 0 is set to 1.0 (Neutral), acting as a placeholder for later fine-tuning on affective datasets.

3 Model Architectures

The repository generates audio via a two-stage cascaded pipeline. The diagrams below have been structured to ensure readability and prevent module overlap.

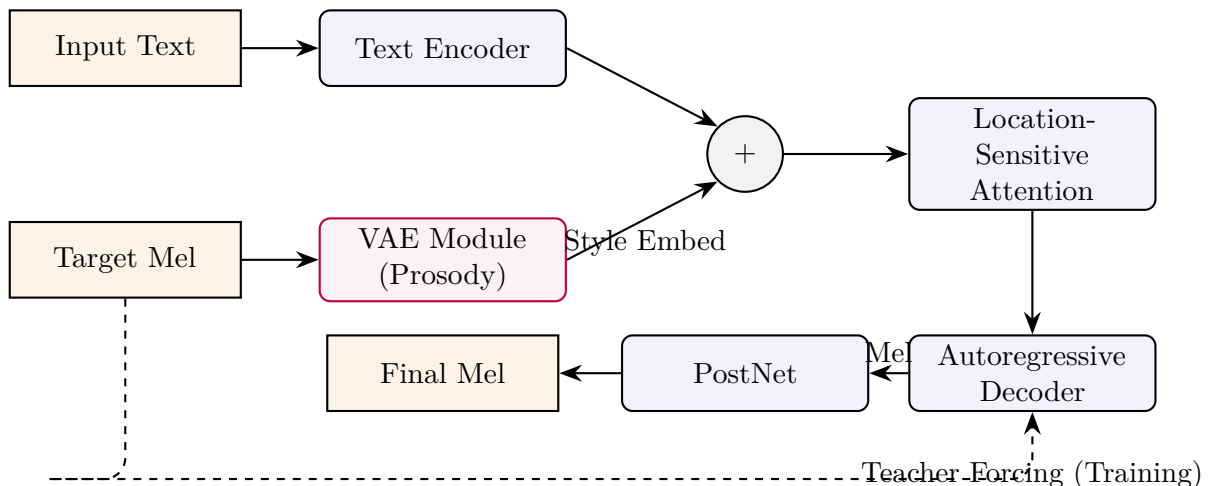
3.1 5.1 Complete System Overview

Text transcripts are converted to character sequences and fed into **Tacotron2-VAE**. Simultaneously, the target audio’s mel-spectrogram is ingested by the VAE module to extract a latent prosody embedding. Tacotron predicts a highly expressive Mel-spectrogram, which is subsequently inverted into a 1D waveform by **WaveGlow**.



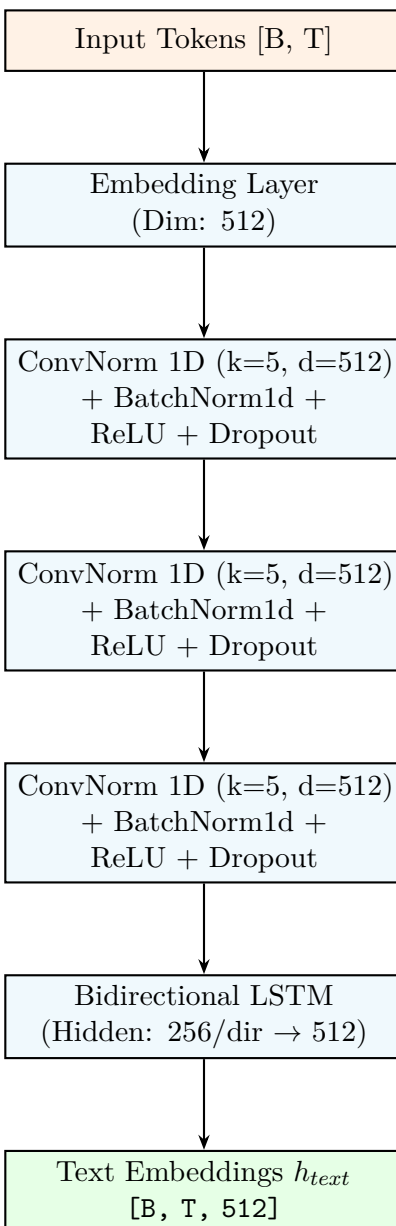
3.2 5.2 Tacotron-VAE Module Overview

Defined in `src/models/tacotron2_vae/model.py`, the model is an autoregressive encoder-decoder.



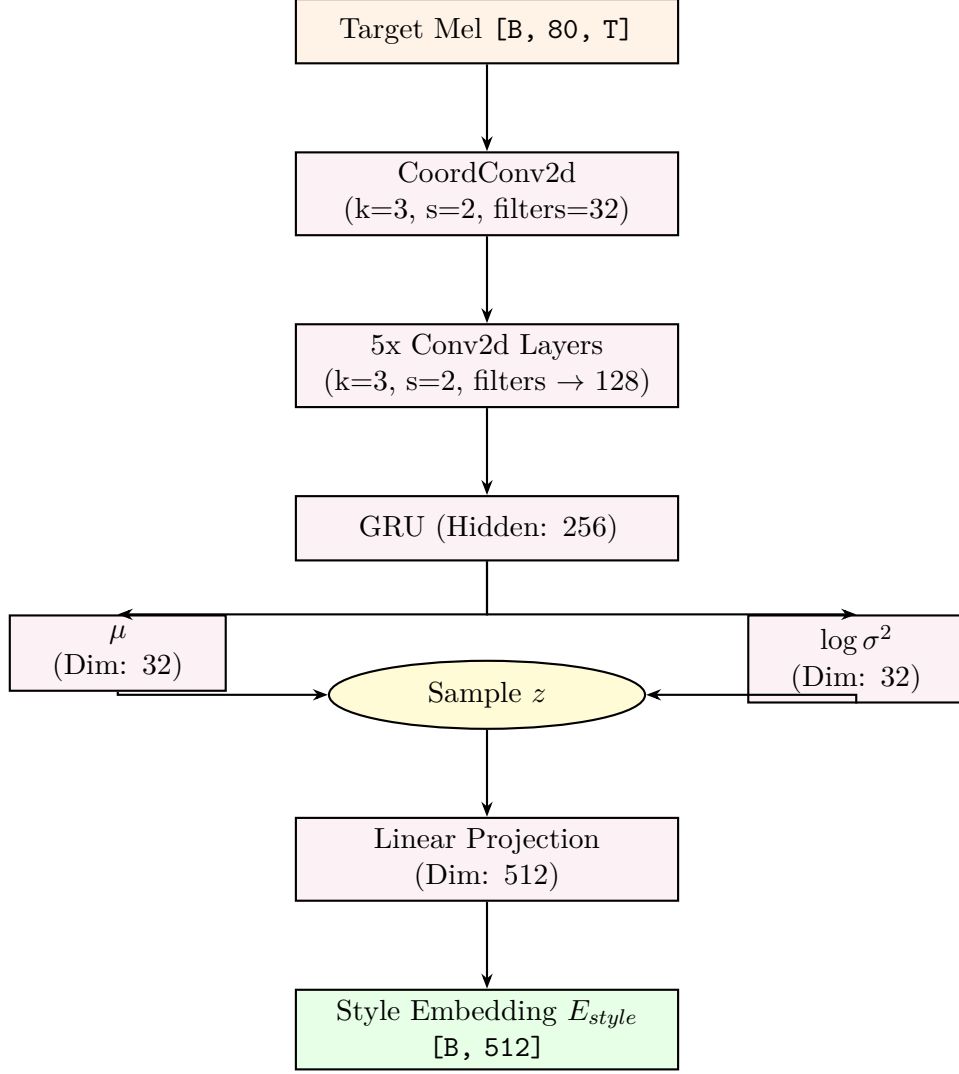
3.3 5.3 Text Encoder Breakdown

The Text Encoder transforms character sequences into robust, context-aware embeddings.



3.4 5.4 VAE (Prosody) Breakdown

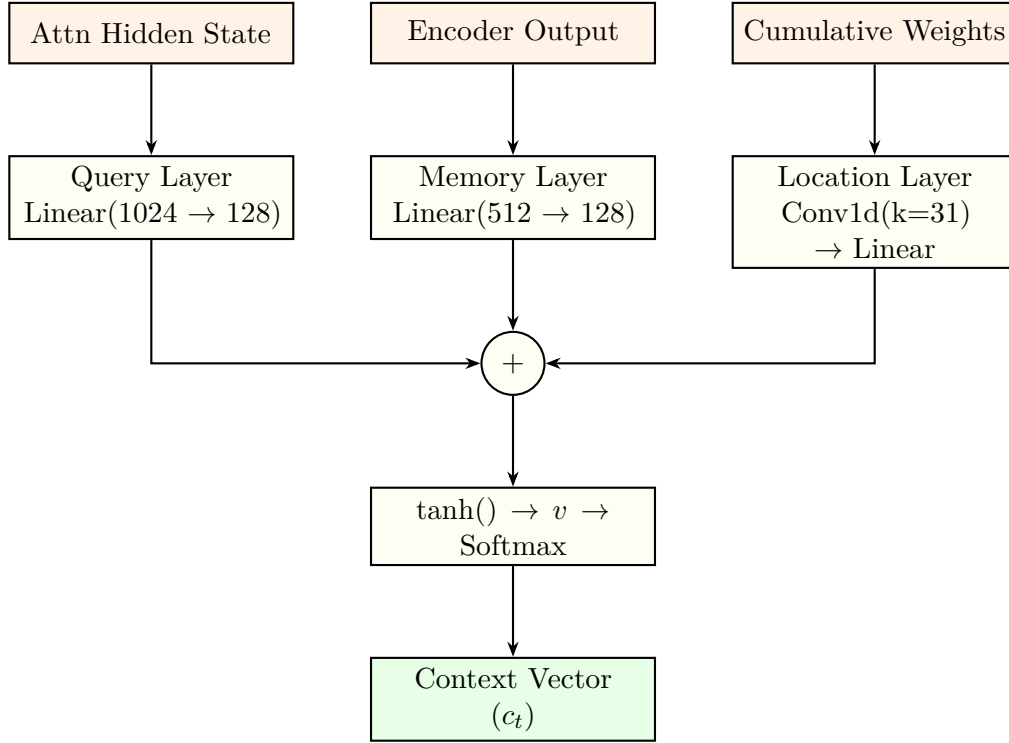
Defined in `modules.py`. **Crucial Distinction:** The implementation defines a class named `VAE_GST`, but it lacks the Multi-Head Attention token bank seen in standard GSTs. Instead, it is a pure Variational Autoencoder that maps the reference mel to a latent distribution, samples z , and projects it linearly to a style embedding.



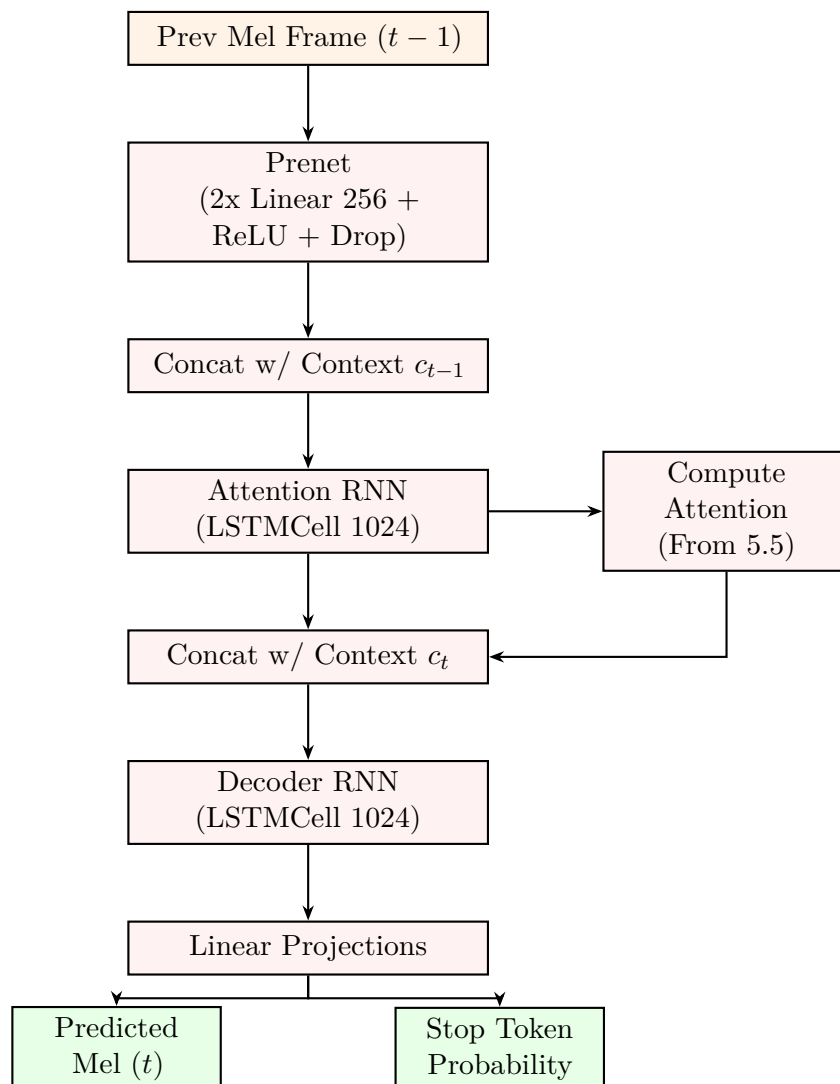
Integration Note: E_{style} is unsqueezed and added to the Text Embeddings: $h_{enc} = h_{text} + E_{style}$.

3.5 5.5 Location-Sensitive Attention Breakdown

The Attention module forces monotonic alignment by convolving over previous attention distributions.

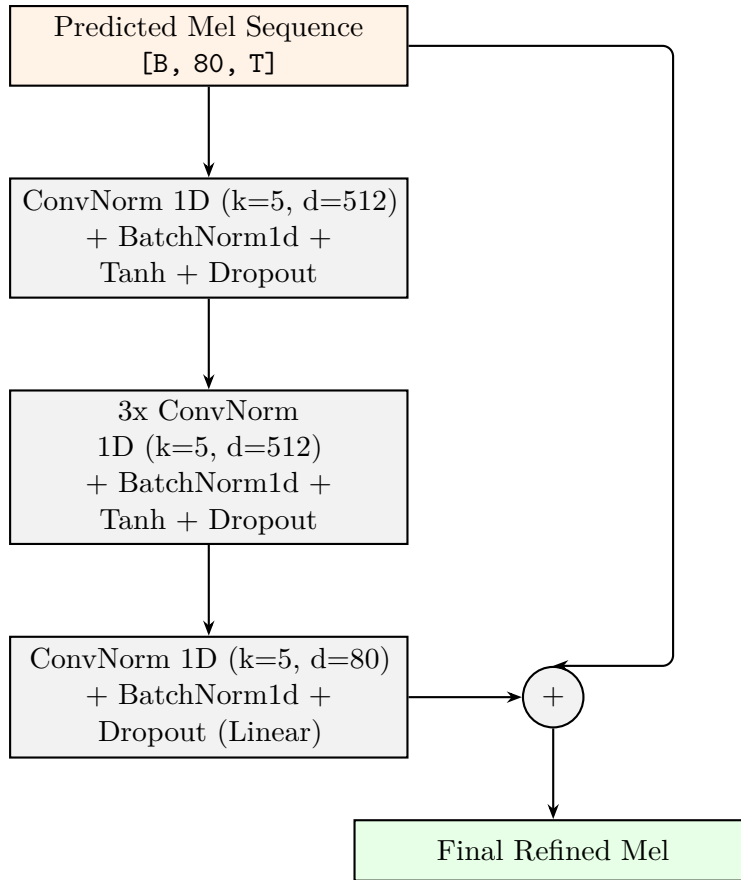


3.6 5.6 Autoregressive Decoder Breakdown



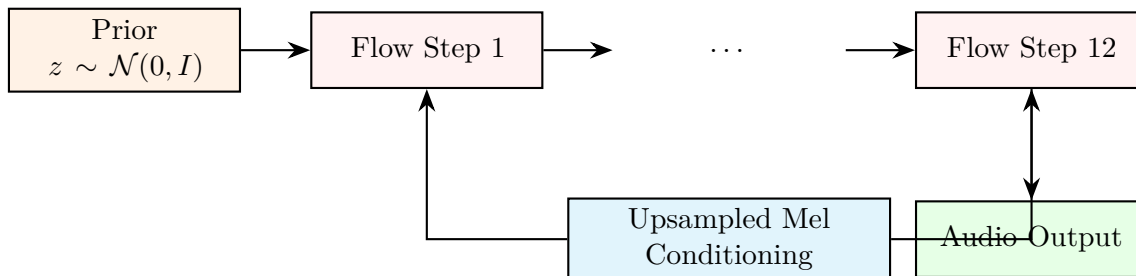
3.7 5.7 PostNet Breakdown

The PostNet operates over the entire predicted sequence to smooth out artifacts.



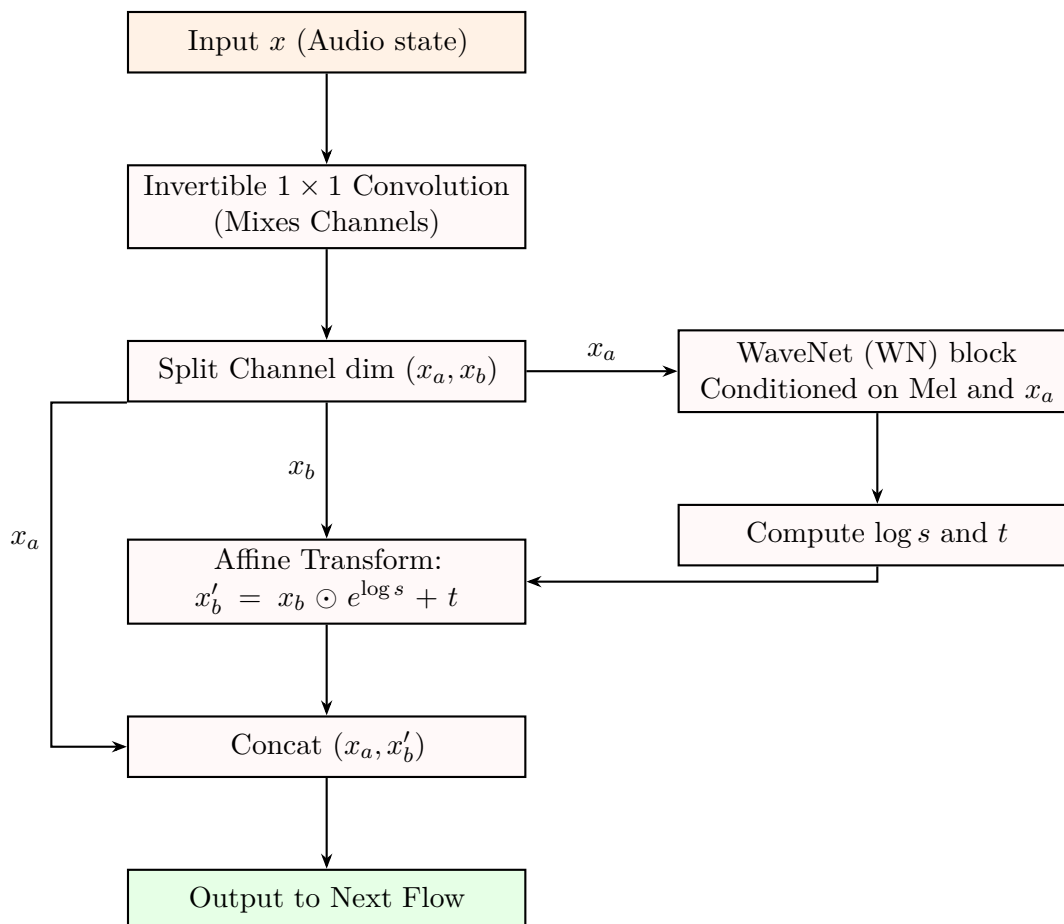
3.8 5.8 WaveGlow Vocoder Overview

WaveGlow (`src/models/waveglow/glow.py`) maps a Gaussian prior into high-fidelity audio conditioned on the synthesized Mel-spectrogram.



3.9 5.9 WaveGlow Flow Step Breakdown

Each flow step is invertible. It consists of an Invertible 1×1 Convolution followed by a WaveNet-based Affine Coupling Layer.



3.10 5.10 Layer-by-Layer Dimension Summary

This section outlines tensor shapes explicitly inferable from `hparams.py` assuming a Batch Size of B and maximum text length T_{text} .

Module	Submodule	Input Shape	Output Shape
Text Encoder	Embedding	$[B, T_{text}]$	$[B, T_{text}, 512]$
	ConvNorm Stack	$[B, 512, T_{text}]$	$[B, 512, T_{text}]$
	Bi-LSTM	$[B, T_{text}, 512]$	$[B, T_{text}, 512]$
VAE (Prosody)	Reference Encoder	$[B, 80, T_{mel}]$	$[B, 256]$
	fc1 (μ), fc2 ($\log \sigma^2$)	$[B, 256]$	$[B, 32]$
	fc3 (Style Embed)	$[B, 32]$	$[B, 512]$
Decoder	Prenet	$[B, 80]$	$[B, 256]$
	Attention Context	-	$[B, 512]$
	Linear Mel Output	$[B, 1024 + 512]$	$[B, 80]$
WaveGlow	Upsampler (ConvTrans)	$[B, 80, T_{mel}]$	$[B, 80, T_{audio}]$
	Output Audio	$[B, T_{audio}, 1]$	$[B, T_{audio}]$

4 Loss Functions

The models are trained independently using custom PyTorch criteria defined directly in their respective submodules.

4.1 Tacotron2-VAE Losses (`losses.py`)

The Tacotron2LossVAE class evaluates three distinct objectives.

4.1.1 1. Reconstruction Loss ($\mathcal{L}_{\text{Recon}}$)

Mean Squared Error is computed both before and after the PostNet to ensure the recurrent network learns the gross spectral envelope while the PostNet refines the residual details.

$$\mathcal{L}_{\text{Recon}} = \|\text{Mel}_{\text{target}} - \text{Mel}_{\text{decoder}}\|_2^2 + \|\text{Mel}_{\text{target}} - \text{Mel}_{\text{postnet}}\|_2^2$$

4.1.2 2. Gate Loss ($\mathcal{L}_{\text{Gate}}$)

Binary Cross Entropy with Logits penalizes incorrect predictions of the stop token.

$$\mathcal{L}_{\text{Gate}} = \text{BCEWithLogits}(\text{gate}_{\text{predicted}}, \text{gate}_{\text{target}})$$

4.1.3 3. Kullback-Leibler Divergence ($\mathcal{L}_{\text{KL}}$)

Regularizes the latent space z to approximate a standard normal distribution $\mathcal{N}(0, I)$.

$$\mathcal{L}_{\text{KL}} = -0.5 \sum_i \left(1 + \log(\sigma_i^2) - \mu_i^2 - \sigma_i^2 \right)$$

4.1.4 4. KL Annealing Strategy

To prevent posterior collapse, a dynamic weight w_{KL} scales the KL loss. The repository allows `logistic`, `linear`, or `constant` scheduling. The default is `logistic`:

$$w_{KL}(\text{step}) = \frac{\text{upper}}{\text{upper} + \exp(-k \times (\text{step} - x_0))}$$

Where upper = 0.2, $k = 0.0025$, and $x_0 = 10000$.

Total Tacotron Objective:

$$\mathcal{L}_{\text{Tacotron}} = \mathcal{L}_{\text{Recon}} + \mathcal{L}_{\text{Gate}} + w_{KL} \cdot \mathcal{L}_{\text{KL}}$$

4.2 WaveGlow Losses (glow.py)

WaveGlow is trained to maximize the exact log-likelihood of the data. Because it uses normalizing flows, the objective relies on the change of variables theorem. The loss is effectively the Negative Log-Likelihood (NLL).

The `WaveGlowLoss` class computes:

$$\mathcal{L}_{\text{WaveGlow}} = \frac{1}{B \cdot C \cdot T} \left(\sum_i \frac{z_i^2}{2\sigma^2} - \sum \log s - \sum \log |\det W| \right)$$

Where:

- z is the latent representation mapped back to the prior.
- σ regulates the variance penalty (default 1.0).
- $\log s$ is the sum of scale logs generated by the affine coupling layers.
- $\log |\det W|$ is the log determinant of the invertible 1×1 convolutions.
- Normalization relies on Batch size (B), Channels (C), and Time length (T).

5 Training Pipeline

Training is orchestrated through execution scripts within `src/training/`.

5.1 Tacotron2-VAE Training

The training entrypoint is `src/training/training-tacotron2-vae/train.py`.

- **Setup:** Reads the `mels_metadata.csv` manifest, builds character symbols, and instantiates `Tacotron2VAEHparams`.
- **DataLoaders:** Instantiates `DatasetLibriSpeechTacotronVAE` and uses `TextMelCollate` to pad batches dynamically. Note that audio waveforms are loaded and converted to Mels iteratively during fetching.
- **Optimizer:** Standard PyTorch Adam optimizer.
- **Loop:** Operates over fixed `epochs`. Gradient clipping is applied (`thresh=1.0`). Checkpoints are saved every `iters_per_checkpoint` interval.

Execution Example:

```
python src/training/training-tacotron2-vae/train.py \  
  --epochs 300 \  
  --batch-size 32 \  
  --learning-rate 1e-3 \  
  --artifacts-dir data/processed/libriSpeech-en-tacotron-vae \  
  --experiment-name v1_tacotron
```

5.2 WaveGlow Training

The training entrypoint is `src/training/training-waveglow/train.py`.

- **Setup:** Parses a `config.json` containing `data_config`, `dist_config`, and `waveglow_config`.
- **Distributed Training:** Heavily leverages PyTorch Distributed Data Parallel (DDP). Initializes process groups across available GPUs.
- **Mixed Precision:** Utilizes NVIDIA Apex (`amp.initialize`) with O1 optimization to drastically reduce VRAM overhead.
- **Loop:** Reads precomputed mel-spectrograms from disks, passes them through the reverse flow, computes the NLL loss, and applies `reduce_tensor` across GPUs.

Execution Example:

```
python -m torch.distributed.launch --nproc_per_node=2 \  
  src/training/training-waveglow/train.py \  
  -c config.json \  
  --checkpoints_dir experiments/waveglow/checkpoints
```

Table 1: Tacotron2-VAE Global Configuration

Category	Hyperparameters
Optimization	Adam ($\text{lr} = 1 \times 10^{-3}$, $\text{weight_decay} = 1 \times 10^{-6}$, $\text{grad_clip} = 1.0$), $\text{batch_size} = 32$
Audio / Features	Sampling Rate=22050 (or 16000), $\text{n_mels} = 80$, $\text{max_wav_value} = 32768.0$
Text Encoding	$\text{symbols_embedding_dim} = 512$, $\text{conv_filters} = 512$, $\text{kernel_size} = 5$
VAE Prosody	$\text{ref_filters} = [32, 32, 64, 64, 128, 128]$, $\text{gru_size} = 256$, $z_latent_dim = 32$, $E = 512$
Attention	$\text{attention_rnn_dim} = 1024$, $\text{attention_dim} = 128$, $\text{loc_filters} = 32$, $\text{loc_kernel} = 31$
Decoder	$\text{decoder_rnn_dim} = 1024$, $\text{prenet_dim} = 256$, $\text{max_steps} = 1000$, $\text{dropout} = 0.1$
PostNet	$\text{postnet_embedding_dim} = 512$, $\text{postnet_kernel_size} = 5$, $\text{convolutions} = 5$

6 Hyperparameters and Configurations

The following values are defined definitively inside `hparams.py`.

Implementation Note regarding Language Support: The configuration declares `text_cleaners=["portuguese_cl...]`. The code defaults to Portuguese normalization heuristics but switches to English text processors (`TextNormalizerEN`) dynamically based on the invoked data loader.